

UNIVERSITETET I OSLO

Maskinlæring i praksis

Pierre Lison
Norsk Regnesentral (NR)

IN1160 – Introduksjon til
maskinlæring (Vår 2026)

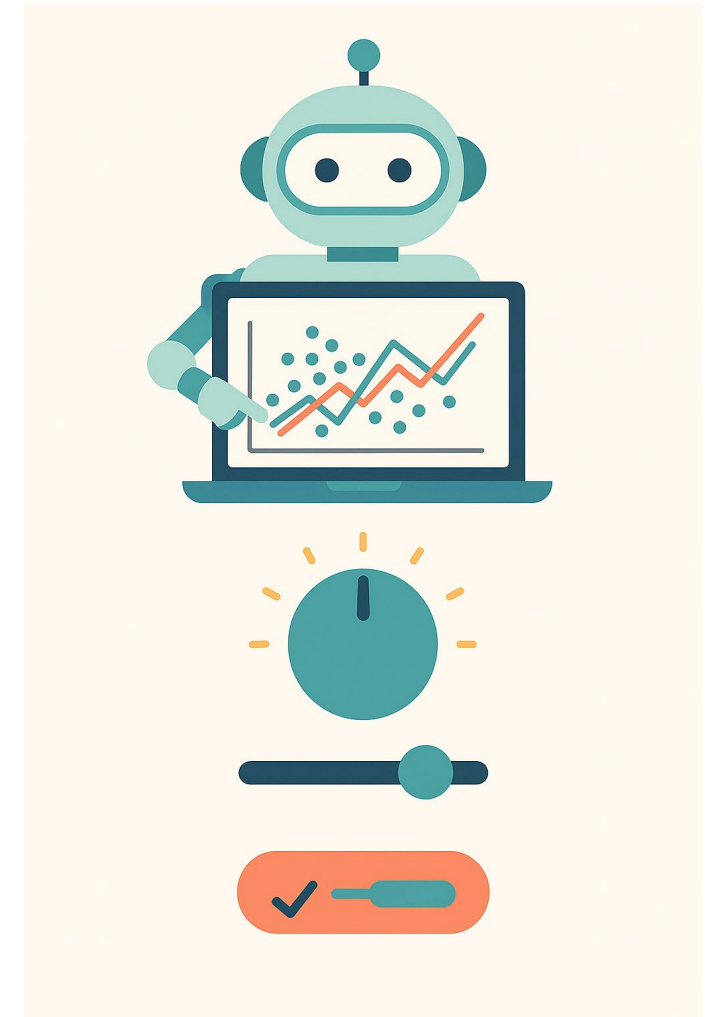
25.03.2026



I dag skal vi snakke om hvordan maskinlæringsteknikker brukes i **praksis**

- Hvordan går vi fra en løst definert problemstilling til en konkret ML-løsning?
- Hvordan velger vi hvilken maskinlæringsmodell som bør brukes?
- Hvilke kriterier bør ligge til grunn for å vurdere kvaliteten på ML-løsningen vår?

+ mange praktiske “tips” og teknikker som kan brukes til å bygge bedre ML-løsninger!



Plan

- Utforskende analyse
- Forbehandling av inputtrekk
- Regularisering
- Modellvalg
- Hyper-parametere
- Oppsummering

Plan

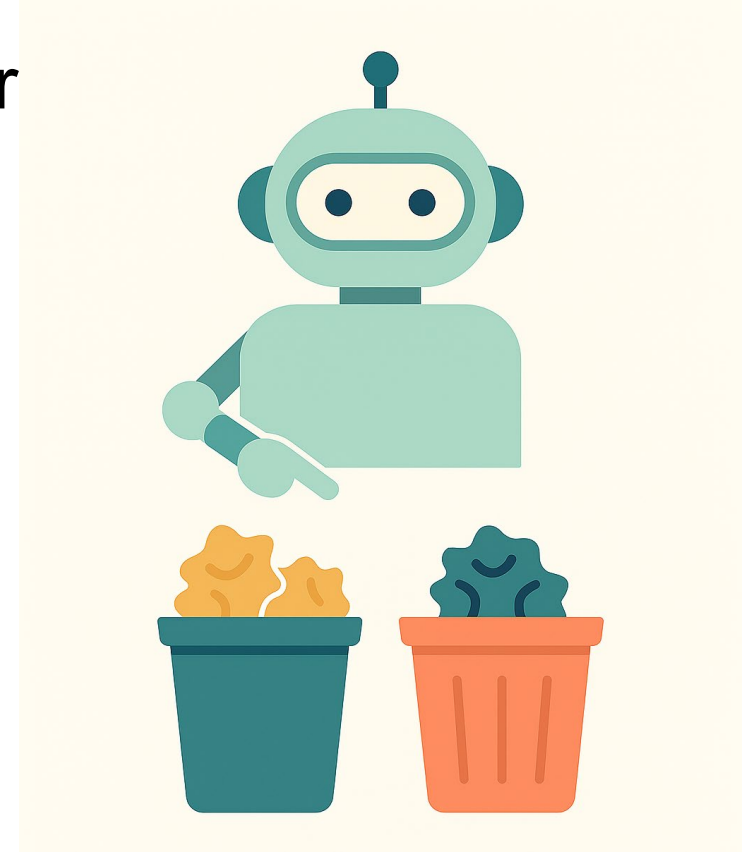
- **Utforskende analyse**
- Forbehandling av inputtrekk
- Regularisering
- Modellvalg
- Hyper-parametere
- Oppsummering

Datainnsikter

Hvis det er én beskjed dere bør sitte igjen med etter denne forelesningen, så er det dette:

Se på dataene dine før du starter å utvikle, trene og sammenligne ML-modeller!

- *Garbage in, garbage out*: det nytter ikke å bruke verdens mest avanserte ML-modell og de kraftigste GPU-er om treningsdataene ikke holder mål!
- Arbeidet med å heve datakvaliteten er minst like viktig (og ofte viktigere!) enn selve modellutviklingen

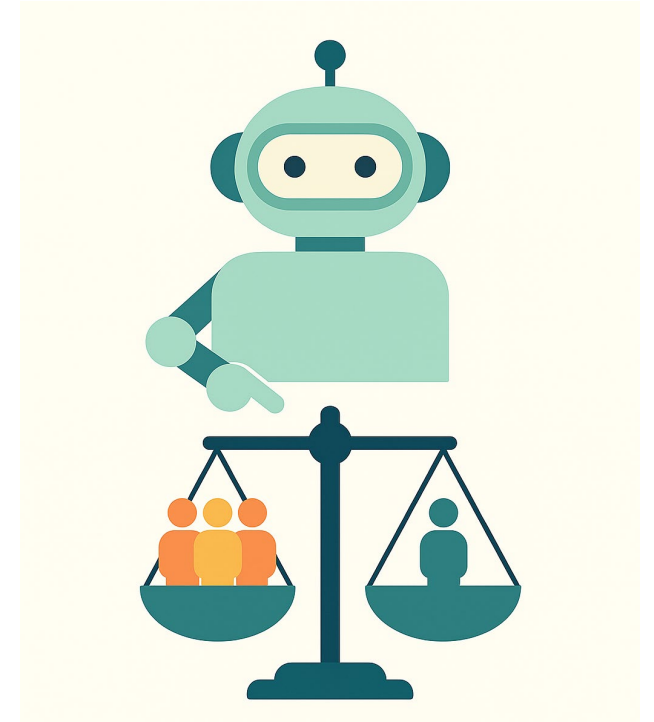


Datainnsamling

- Første spørsmål å stille seg: **hvordan ble dataene samlet inn?**
- Ulike *statistiske skjevheter* kan nemlig oppstå under innsamlingsprosessen:
 - **Utvalgsskjevhet** (*selection bias*): Er dataene representative for alle «enheter» vi ønsker å jobbe med (også kalt *populasjonen* i statistikk)?
 - **Måleskjevhet** (*measurement bias*): Er dataene preget av målefeil eller rapporteringsfeil, f.eks. upresise eller inkonsekvente annoteringer?
 - **Historisk skjevhet** (*historical / temporal bias*): Er dataene utdaterte eller ikke helt representative for dagens situasjon?
 - **Frafallskjevhet** (*attrition / survivorship bias*): Er dataene bevisst eller ubevisst «filtrert», slik at en del av populasjonen falt ut av innsamlingen og mangler i datasettet?
 - Og mange andre!

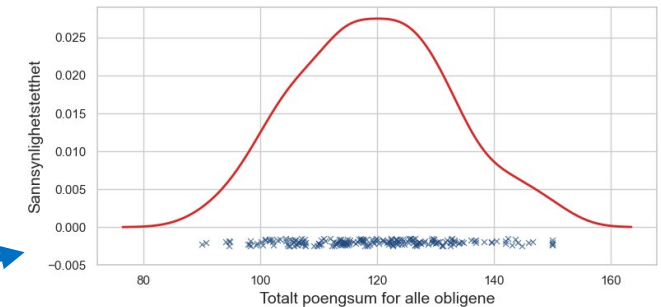
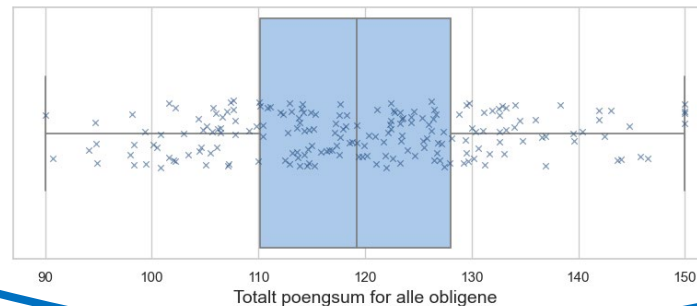
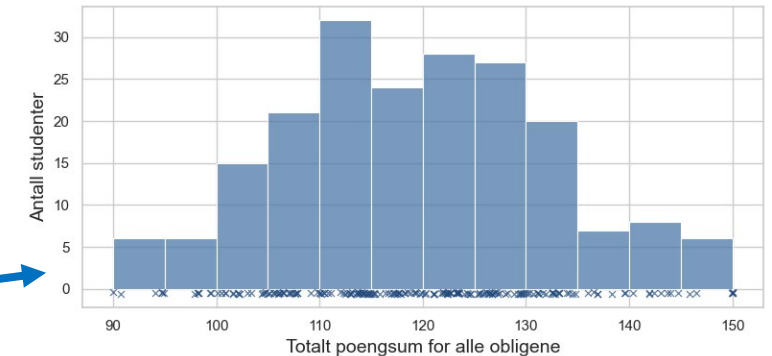
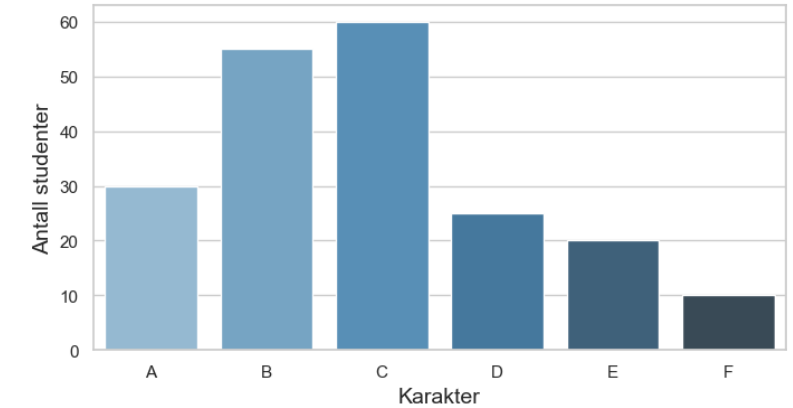
Datainnsamling

- Hvorfor er statistiske skjevheter viktige?
 - Fordi skjevheter gjør at modellens ytelse på datasettet ikke nødvendigvis vil overføres til virkeligheten
 - Fordi skjevheter også kan ha *etiske konsekvenser* når data omhandler mennesker (mer om det om noen uker)
- Hva kan vi gjøre?
 - *Kvalitetssikre* datapunktene (er det noen åpenbare feil?)
 - I alvorlige tilfeller av utvalgsskjevhet kan vi *rebalansere datasettet* ved å gi større vekt til underrepresenterte grupper
 - Ved behov: samle inn et *større / bedre datagrunnlag*



Univariat analyse

- Spørsmål nr. 2: **hvordan ser datatrekkene ut?**
- Vi starter med å ta *hvert trekk for seg* og se hvordan dets verdier fordeler seg
- For kategoriske trekk kan vi tegne et *søylediagram* for å vise hvor ofte hver verdi forekommer.
- For numeriske trekk har vi flere alternative visualiseringer:
 - Histogrammer
 - Boksplott
 - Sannsynlighetsstetthet
 - Osv.



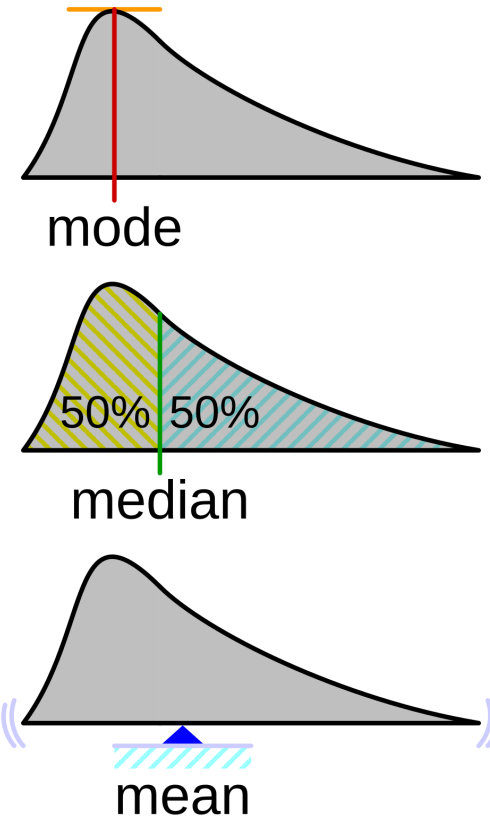
Univariat analyse

Tre nøkkeltall i statistikk som peker på «senteret» i fordelingen:

1. **Gjennomsnittet** $\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$
2. **Medianen**, som er “middelverdien” i en tallrekke. La $x_{(1)}, \dots, x_{(n)}$ betegne den *sorterte* tallrekken for $\mathbf{x}$, hvor $x_{(1)}$ tilsvarer den minste verdien og $x_{(n)}$ den høyeste. Da er medianen definert som

$$\begin{cases} x_{(\frac{n+1}{2})} & \text{hvis } n \text{ er oddetall} \\ \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2} & \text{hvis } n \text{ er partall} \end{cases}$$

1. **Typetallet** (eng. *mode*), er verdien som forekommer oftest



Source: https://commons.wikimedia.org/wiki/File:Visualisation_mode_median_mean.svg

Medianen er altså utformet slik at 50% at tallene i rekken $\mathbf{x}$ havner under den og 50% over den.

Mentimeter

- Gitt ett trekk med følgende verdier:

$[2 \quad -1 \quad 0.5 \quad 3 \quad 2 \quad -1.5 \quad -4 \quad 79]$

- Beregn gjennomsnittet, medianen og typetallet

Univariat analyse

Spredning av verdiene kan beskrives gjennom ulike statistiske målinger:

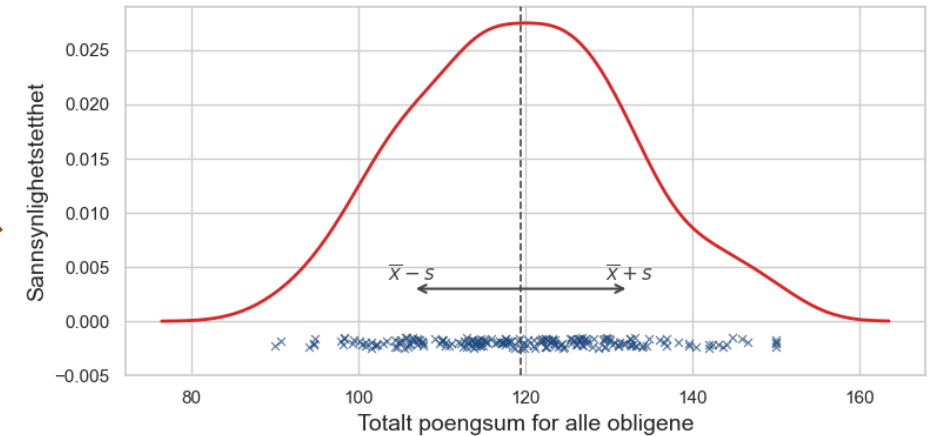
1. **Minimum- og maksimumsverdier**
2. **Standardavviket** (eng. *standard deviation*), som måler hvor mye verdiene i snitt avviker fra gjennomsnittet :

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Hvorfor $n - 1$ og ikke n i nevner? Formelen her tilsvarer det såkalte «empiriske» eller «korrigerte» standardavviket, som tar høyde for at verdiene $x_1, \dots, x_n$ kun er en stikkprøve fra en større populasjon.

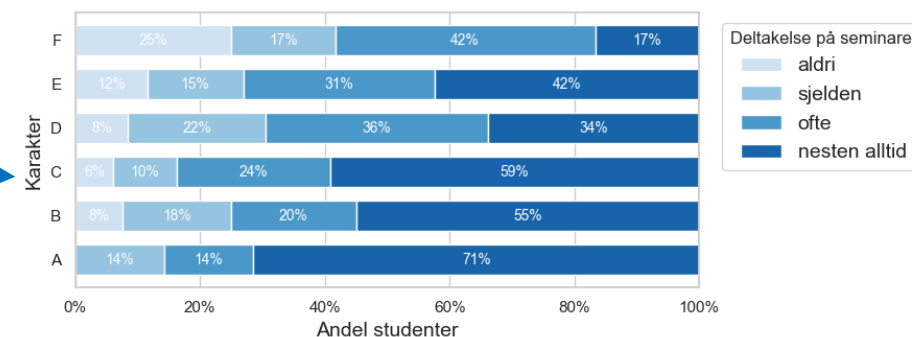
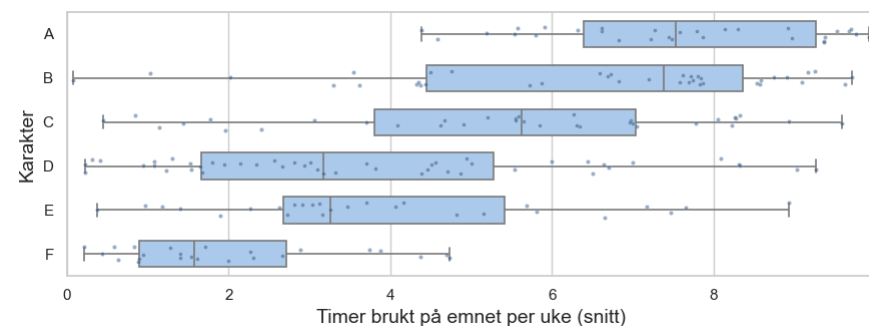
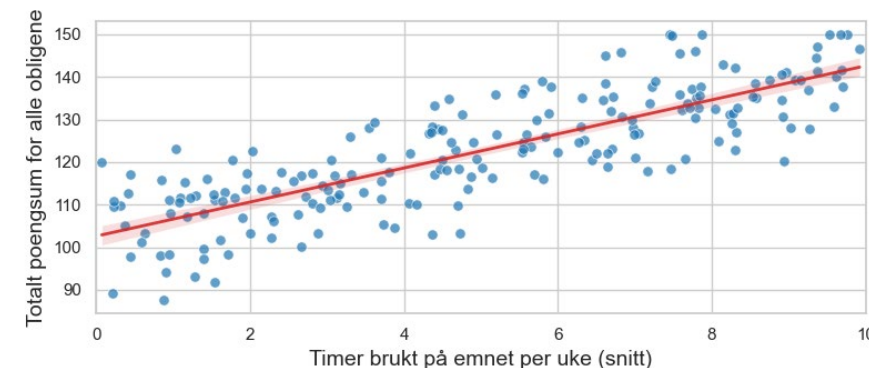
Når verdiene følger en normalfordeling vil

- cirka 68% av datapunktene ligge innen $\pm 1 s$ av snittet
- cirka 95% av datapunktene ligge innen $\pm 2 s$ av snittet
- cirka 99.7% av datapunktene ligge innen $\pm 3 s$ av snittet



Multivariat analyse

- Også nyttig å undersøke hvordan trekkene
 - korrelerer med **målvariabelen** (responsen)
 - samspiller med **hverandre**
- Ulike visualiseringer:
 - Hvis de to variablene er numeriske:
spredningsdiagrammer (eng. *scatter plots*)
 - Med hybride (numeriske og kategoriske) variabler kan vi bruke f.eks. *boksplotter*
 - Og for to kategoriske variabler kan vi bruke f.eks. *stabledede søylediagrammer*
- Slike visualiseringer kan hjelpe oss å avdekke ulike sammenhenger



Praktiske dataspørsmål

1. Er trekkene representert i *riktig format*?
 - For eksempel numeriske verdier representert som strenger

→ Hvis det ikke er tilfellet: sørg for at formatet er riktig 😊
2. Har noen datapunkter trekk hvor noen trekk mangler *verdier*?
 - Ofte fordi verdien for trekket ikke ble «observert» for punktet og dermed er ukjent
 - Mangelfulle verdier er ofte i praksis representert som *NaN* (Not a Number)

→ Hvis svaret er ja må du enten:

 - a) Fjerne *datapunktene* med manglende verdier
 - b) Utelate *trekkene* med manglende verdier
 - c) Imputere plausible verdier når den mangler (*missing data imputation*)
 - d) Ta i bruk ML-modeller som kan trenes på data med manglende verdier

Praktiske dataspørsmål

3. Ser vi noen tydelige *utliggere*?

- Det finnes en rekke metoder for å automatisk avdekke utliggere i et datasett (*outlier detection*)

→ Utliggere som skyldes datafeil eller lignende kan ofte ignoreres, eller behandles som manglende verdier (hvis det er snakk om anomalier knyttet til ett trekk)

→ «Ekte» utliggere som tilsvarer reelle observasjoner bør derimot undersøkes nærmere.

4. Er noen trekk *overflødige* eller *duplikater* av hverandre?

- Overflødige trekk = trekk som er tett korrelert med andre trekk, eller med overlappende innhold

→ Disse bør droppes om de ikke bidrar med ny informasjon

→ Men «kraftigere» ML-modeller, f.eks. ensembler av beslutningstrær eller dype nevrale nettverk, er ofte ganske robuste mot overflødige trekk, i hvert fall om modellen er godt *regularisert* (mer om det om noen slides)

Praktiske dataspørsmål

5. Bør vi konstruere *nye trekk* basert på de eksisterende?

- For eksempel forholdstall x_i/x_k , logaritmiske skalaer $\log x_i$, osv.
- Slike avledede trekk kan brukes til å fange opp samspill mellom trekk, eller ikke-lineære korrelasjoner mellom trekk og responsen

→ Avledede trekk er spesielt nyttige når modellen er lineær (mer sofistikerte ML-modeller vil kunne automatisk «fange opp» slikt samspill, gitt nok treningsdata)

Plan

- Utforskende analyse
- **Forbehandling av inputtrekk**
- Regularisering
- Modellvalg
- Hyper-parametere
- Oppsummering

Kategoriske trekk

- Mange ML-modeller kan bare ta numeriske trekk som input
- Hva om jeg har kategoriske trekk?
 - Da må vi konvertere disse til tall
 - `is_blacklisted` kan f.eks. omskrives som `False = 0` og `True = 1`
- Ok, men hva om jeg har > 2 mulige verdier?
 - yrke kan f.eks. være [snekker, ingeniør, lærer, frisør, sykepleier, ...], med k mulige verdier.
 - Da kan man bruke såkalt *one-hot encoding* til å omskrive variabelen som en vektor av lengde k med 0 overalt bortsett fra én celle som har verdi 1:

yrke = "frisør" $\rightarrow$ [0 0 0 1 0 ...]



Lengde k (antall mulige verdier)

Kategoriske trekk

- One-hot encoding øker antall inputtrekk
 - Hvert kategorisk trekk blir til k binære trekk
 - Risiko for *overtilpasning* om vi ikke har nok treningsdata i forhold til antall trekk!
→ I dette tilfellet bør vi *slå sammen* beslektede verdier (aggregering), f.eks. ved å samle yrker i mer overordnede grupper (håndverker, helsepersonnel, osv.)
- Noen kategoriske variabler har også en naturlig «orden»
 - F.eks. karakterer A-F, eller skalaer som «veldig liten», «liten», «stor», «veldig stor»
- Slike såkalte *ordinalvariabler* kan konverteres til tall i $[1, k]$
 - 1 vil i så fall tilsvare minimumsverdien i skalaen, og k maksimumsverdien

Merk: det finnes metoder som kan automatisere slik aggregering, f.eks. ved å utnytte det såkalte “hashing trick” eller forhåndstrengte ordvektorer. Dette er ikke en del av pensumet, men dere kan gjerne google det om dere er interessert 😊

Skalering

- Mange maskinlæringsmodeller forutsetter også at trekkene befinner seg innenfor omtrent samme størrelsesorden.
- *Hvorfor det?* Fordi trekk som er betydelige større (positivt eller negativt) enn andre kan ende opp med å «dominere» modellen
- Dette kan løses ved å *skalere* eller *normalisere* trekkene
- En enkel metode er å regne ut gjennomsnittet $\bar{x}$ og standardavviket s for trekket x , og deretter lage det nye trekket:

$$x' = \frac{x - \bar{x}}{s}$$

Merk: per definisjon vil det nye trekket x' ha et gjennomsnitt = 0 og et standardavvik = 1.

Mentimeter

- Gitt de følgende verdiene:

[1002 986 998 1014]

- Regn ut de normaliserte verdiene:

$$\mathbf{x}' = \frac{\mathbf{x} - \bar{x}}{s}$$

Husk:

Gjennomsnittet $\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$

(Ukorrigert) standardavviket $s = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$

Plan

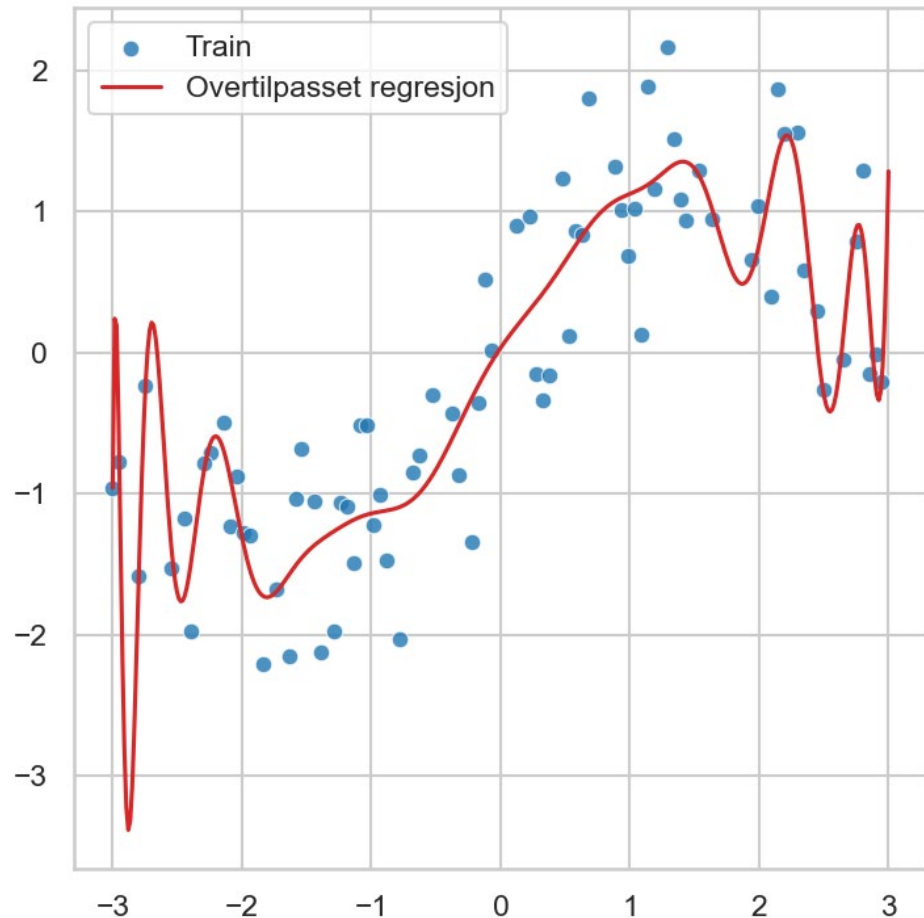
- Utforskende analyse
- Forbehandling av inputtrekk
- **Regularisering**
- Modellvalg
- Hyper-parametere
- Oppsummering

Overtilpasning

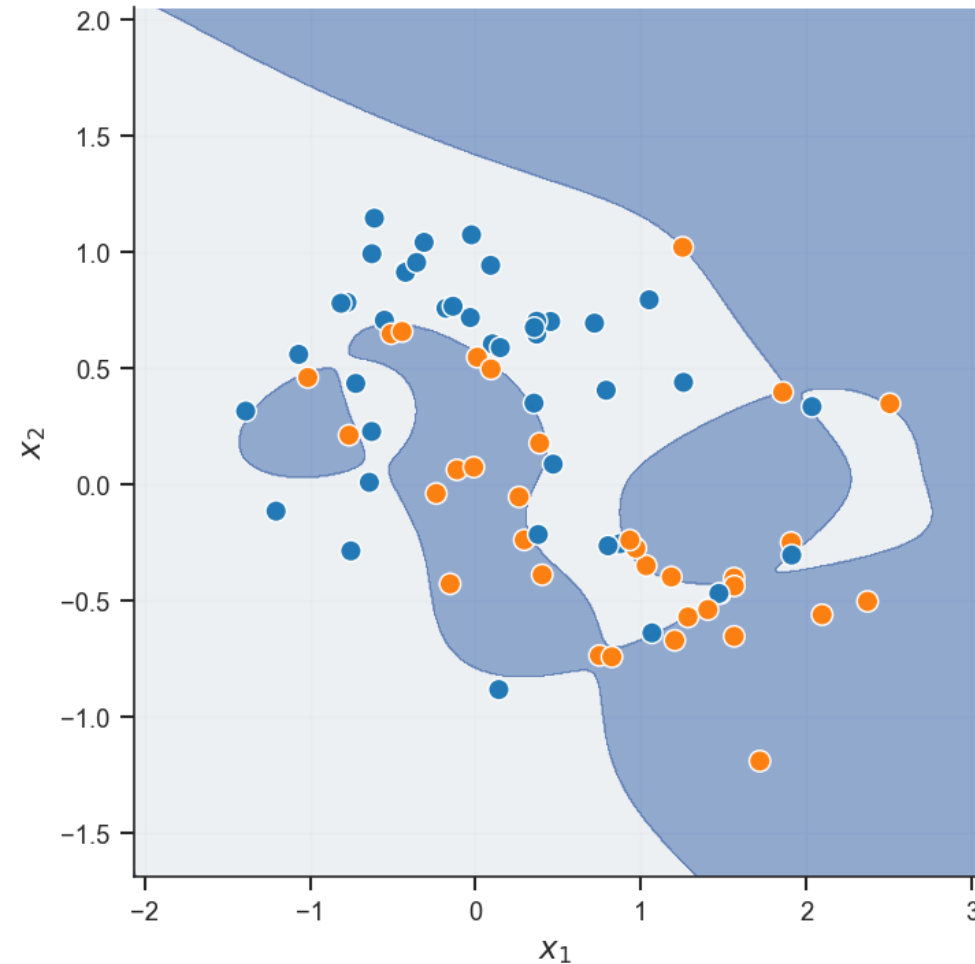
- Vi trener vanligvis ML-modeller ved å minimere deres tap på *treningssettet*
- ... men målet vårt er egentlig å få en modell som yter best mulig (=med minst mulig tap) på nye datapunkter
- **Overtilpasning** (*overfitting*) skjer når en ML-modell fungerer veldig bra på treningssettet, men dårlig på nye data
 - Mao. når det gjennomsnittlige tapet på treningsdata (*training loss*) er vesentlig lavere enn det gjennomsnittlige tapet på datapunkter utenfor treningssettet, f.eks. på et valideringssett (*validation loss*)

Overtilpasning

... i regresjon



... i klassifikasjon



Overtilpasning

- En vanlig grunn til overtilpasning er at ML-modellen har for mange parametere i forhold til tilgjengelig treningsdata
 - Da kan vi ende opp med en modell som perfekt har “memorert” datapunkter i treningssettet men *generaliserer* dårlig til nye punkter
- Heldigvis kan vi bruke såkalte **regulariseringsteknikker** til å redusere risikoen for overtilpasning
- Vi skal nå gå gjennom noen av disse teknikkene

Eksplisitt regularisering

Hvorfor? Fordi “enklere” modeller har mindre risiko for overtilpasning. Vi skal straks se nærmere på hva vi mener med “enklere”

- En vanlig måte å regularisere ML-modeller er å omdefinere *tapet* ved å legge til et ledd som gir et fortrinn til “enklere” modeller
- Så langt har vi skrevet det (gjennomsnittlige) tapet på et treningssett som

$$L = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)})$$

$\hat{y}^{(k)}$ er prediksjonen fra modellen basert på inputtrekkene $\mathbf{x}^{(k)}$

$y^{(k)}$ er fasitverdien i treningssettet for trekkene $\mathbf{x}^{(k)}$

Husk: Når vi trener en ML-modell forsøker vi å finne verdiene for modellparametere som *minimerer* dette tapet L på treningssettet. Dette gjøres ofte gjennom en variant av gradientnedstigning

l er tapsfunksjonen som måler avviket mellom $\hat{y}^{(k)}$ og $y^{(k)}$. Vi har allerede sett flere eksempler av tapsfunksjoner, bl.a. kvadraftfeil og kryssentropi.

Eksplisitt regularisering

λ uttrykker *regulariseringsstyrken*, altså hvor viktig regulariseringen bør være i forhold til det gjennomsnittlige tapet på treningspunktene.
 λ er en typisk *hyper-parameter*

- Den meste kjente metoden er **L2-regularisering**, som brukes til å “straffe” modeller med høye parameterverdier:

$$L_{\text{reg}} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)}) + \lambda \sum_{i=0}^p \beta_i^2$$

p er antall parametere i modellen (hvis lineær modell: antall trekk + 1)

Én parameter (eller “koeffisient”) i modellen

- Hvorfor er L2-regularisering nyttig ?* Fordi høy parameterverdier betyr at modellen blir veldig sterkt påvirket av ett bestemt trekk, noe som er uheldig
- Hvordan fungerer det ?* Hvis en parameter β_i har en høy (positive eller negative) verdi vil β_i^2 bli enda høyere, og dermed også tapet L_{reg} . Modeller med lavere parameterverdier vil dermed bli foretrukket.

Eksplisitt regularisering

- En annen regulariseringsstrategi er **L1-regularisering**:

$$L_{\text{reg}} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)}) + \lambda \sum_{i=0}^p |\beta_i|$$

Absoluttverdien av parameteren

- L1 regularisering brukes til å gi et fortrinn til *sparsomme* modeller, altså modeller som ikke tar i bruk flere parametere enn nødvendig
 - L1 skyver de unødvendige parameterne mot nær-null verdier
- Merk: L1 og L2 regularisering kan brukes samtidig!

Mentimeter

- **Spørsmål:** hva vil skje med modellen om vi håndhever en veldig sterk regularisering, altså f.eks. med en verdi $\lambda = 1000$?

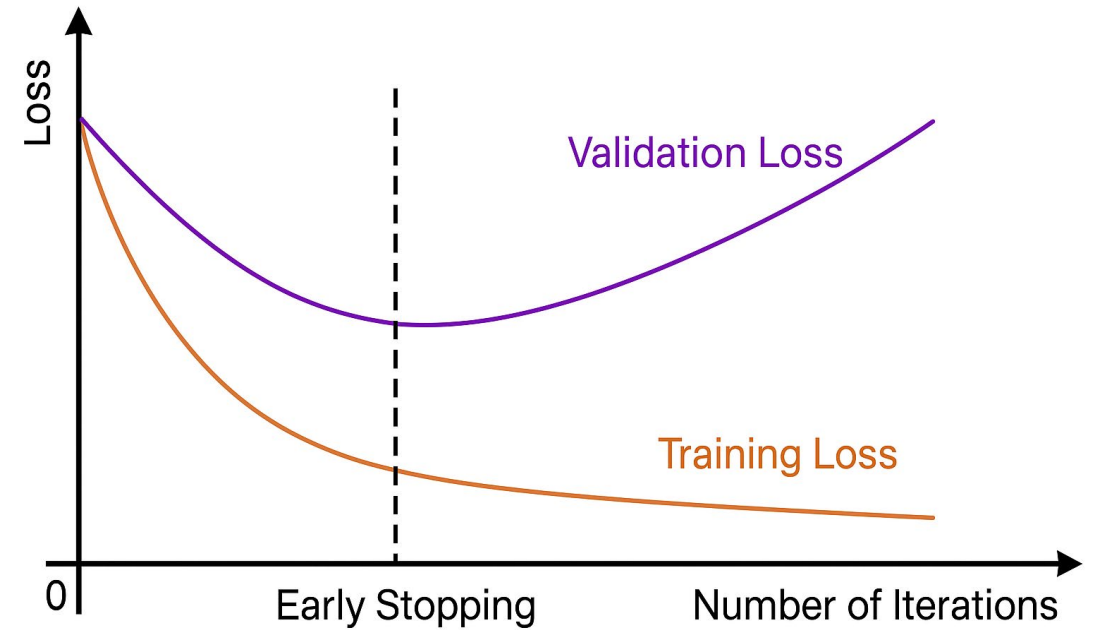
Husk:

$$\text{L2-regularisering: } L_{\text{reg}} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)}) + \lambda \sum_{i=0}^p \beta_i^2$$

$$\text{L1-regularisering: } L_{\text{reg}} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)}) + \lambda \sum_{i=0}^p |\beta_i|$$

Implisitt regularisering

- Det finnes flere andre måter å bekjempe overtilpasning på
- For eksempel å avslutte treningen før modellen er helt konvergjert til en endelig løsning (*early stopping*)
- For nevrale nettverk kan man også hoppe over en del nevroner i nettverket (valgt ut på tilfeldig basis) for hver batch (*dropout*)
- Og for beslutningstrær kan man f.eks. begrense maks tredybde, slik vi så forrige uke



Plan

- Utforskende analyse
- Forbehandling av inputtrekk
- Regularisering
- **Modellvalg**
- Hyper-parametere
- Oppsummering

Modellvalg

- La oss si du blir bedt om å utvikle en ML-modell for å automatisere en bestemt oppgave, basert på et datasett
- Hvordan går du fram for å velge ML-modellen som skal brukes?
- Ingen “*one size fits all*”. Flere faktorer må tas hensyn til:

Skal modellen
trenes basert
på en
fasitverdi?

Hvordan ser
datasettet ut?

Hvor komplisert
bør modellen
være?

Hvilke kriterier
bør modellen
vurderes på?

Skal ML-modellen trenes basert på en *fasitverdi*?

Ja (veiledet læring)

Nei (uveiledet læring)

Er output

- et tall (*regresjon*)
- en klasse (*klassifikasjon*)
- en preferanse mellom alternativer (*rangering*)
- noe mer komplisert, f.eks. en graf, et tre eller en sekvens (*strukturert prediksjon*) ?

Skal modellen

- gruppere sammen datapunkter (*klyngeanalyse*),
- avdekke *utliggere/anomalier*
- “komprimere” datapunktene (*dimensjonsreduksjon*)
- estimere *tettheten* til en eller flere kontinuerlige variabler?

Hvordan ser datasettet ut?

- Det finnes også svært mange ML-modeller som er spesielt tilpasset for bestemte datatyper eller outputs
- For eksempel for bilder, lyd eller tekst, hvor man har mange spesialiserte modeller, i dag typisk basert på forhåndstreinte grunnmodeller
- Eller for *tidsserier* (predikere fremtiden basert på fortiden), hvor man også har dedikerte metoder



Hvor komplisert bør modellen være?

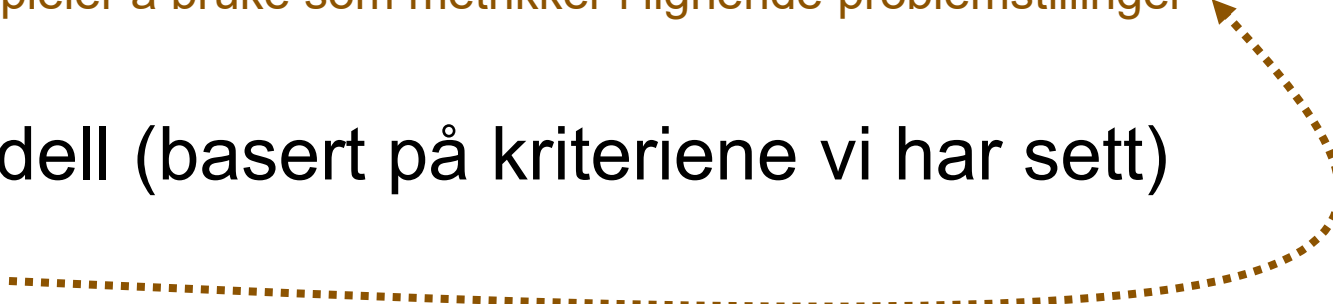
- En god tommelregel: *start enkelt og kompliser etter behov*
- Mer komplekse modeller slik som nevrale nettverk kan fange mer avanserte mønstre ...
- ... men er også lettere å overtilpasse, og vanskeligere å feilsjekke
- Når man eksperimenter med ulike ML-modeller bør man sammenligne resultatene med en “baseline” som viser hva kan oppnås med en enklere metode
 - Hvis forskjellen er minimal → bedre å bruke den enkle metode

Hvilke kriterier bør modellen vurderes på?

- Modellens ytelse på et valideringssett er selvsagt et viktig kriterium
- Men andre kriterier kan være like viktige:
 - Hvor *rask* kan ML-modellen kjøre?
 - Er modellen lett å *tolke* og forklare?
 - Kan modellen handle f.eks. manglende verdier? Millioner av trekk? Kategoriske trekk?
 - Hvor «robust» er modellen mot atypiske datapunkter?

Fremgangsmåte

Hvordan vet vi hvilke metrikker passer? Et godt utgangspunkt er å undersøke (f.eks. i fagartikler) hva andre pleier å bruke som metrikker i lignende problemstillinger



1. Velg en passende ML-modell (basert på kriteriene vi har sett)
2. Velg passende metrikker
3. Kjør evalueringen, enten med kryssvalidering eller et valideringssett
 - Kryssvalidering bør foretrekkes når datamengden er liten
4. Analyser resultatene, og utforske hva som kunne gjøres bedre
 - Enten ved å bruke en annen ML-modell, eller ved å justere ulike hyperparametere (mer om det om noen minutter)
5. Når du er helt ferdig: sist evaluering på et utstilt testsett

Plan

- Utforskende analyse
- Forbehandling av inputtrekk
- Regularisering
- Modellvalg
- **Hyper-parametere**
- Oppsummering

Hyper-parametere

Hvorfor hyper-parameter? For å skille disse fra de “vanlige” parameterne (f.eks. vektene i regresjon), som estimeres basert på treningssettet



- De fleste ML-modeller kommer med noen såkalte «hyper-parametere» som må fastsettes
 - Slik som antall naboer k i k -NN, antall noder i hvert skjulte lag i nevralt nettverk, eller maksdybden i beslutningstrær
 - Treningsalgoritmen har også hyper-parametere, slik som læringsraten, regulariseringsstyrken λ , eller størrelsen på batch
 - Hyper-parameterne kan ha en stor påvirkning på hvor godt modellen fungerer
- Hvordan kan vi finne gode verdier for disse?

Hyper-parametere

- *Naiv tilnærming*: ta hver hyper-parameter separat, test ut ulike verdier på valideringssettet / kryssvalidering, og velg hva som gir best resultat
- Merk at det kan være en regnekrevende prosess, da hver test krever å trene modellen på nytt!
- Problemet er at hyper-parametere er ofte *korrelert med hverandre*: en læringsrate kan fungere veldig bra med en bestemt batchstørrelse, men dårlig med en annen
- → Vi må teste ut *kombinasjoner* av hyper-parameterverdier

Hyper-parametere

- La oss si vi har to hyper-parametere: læringsraten η og batchstørrelsen b , som kan ta følgende mulige verdier:

$$\eta \in \{0.01, 0.02, 0.05, 0.001, 0.005, 0.0001\}$$

$$b \in \{1, 2, 4, 8, 16, 32, 80\}$$

- Antall mulige kombinasjoner for (η, b) er dermed $6 \times 7 = 42$
- En enkel metode: teste ut alle 42 kombinasjoner – men vi må da trene 42 modeller og teste hver av dem på valideringssettet!
- Vi kan også ta *et tilfeldig utvalg* av kombinasjoner og bare teste disse – men vi kan da gå glipp av den “beste” kombinasjonen

Hyper-parametere

- Heldigvis finnes det optimeringsverktøy som automatisk kan søke på de beste verdiene for hyper-parametere
- **Intuisjon:** optimering starter med tilfeldige verdier, teste disse ut og gradvis avdekke delen av søkerommet som er mest «lovende» iht. resultater på valideringssettet
- Et kjent Python verktøy for slik optimering er `optuna`

AutoML

- Vi kan ta ideen om automatisk optimering av hyperparametere et stykk lenger, og optimere **alt**
 - ... altså ikke bare hyperparametere, men også valget av ML-modell, ensembling, forbehandling av inputtrekk, osv.
 - Dette kalles *AutoML*
 - Et kjent Python-bibliotek for AutoML: `autogluon`
 - AutoML kan hjelpe oss få best mulige resultater...
- ... men gjør det vanskeligere å forstå hva modellen egentlig gjør!

Plan

- Utforskende analyse
- Forbehandling av inputtrekk
- Regularisering
- Modellvalg
- Hyper-parametere
- **Oppsummering**

Oppsummering

Noen viktige tips for å få robuste og pålitelige ML-løsninger:

1. *Se på dataene dine!* Utforsk trekkene, sjekk ut hvordan verdiene fordeler seg, avdekke mulige utliggere, finn ut om noen punkter mangler verdier, osv.
2. Husk å *forbehandle* trekkene dine: konvertering av kategoriske trekk, skalering av numeriske trekk, osv.
3. Overfitting kan bekjempes ved å bruke *regularisering*, enten med L1 og L2-regularisering eller andre metoder

Oppsummering

Noen viktige tips for å få robuste og pålitelige ML-løsninger:

4. Reflekter over *hvilke kriterier* er viktige for akkurat din problemstilling. Ytelsen betyr ikke alt!
(må løsningen kjøre raskt? Være forklarbar?)
4. *Hyper-parametere* bør også justeres for å få optimalt ytelse
5. Hvis du er usikker på hvilke løsning er best: prøv *AutoML*!